# HAI MODULE AI CỦA HỆ THỐNG TASKFLOW — CÁC Ý CHÍNH

*Bản tóm tắt dùng để báo cáo*

## Mở đầu — nói trong 30 giây

Em bổ sung vào hệ thống quản lý công việc TaskFlow **hai module thông minh giải quyết hai đầu của cùng một quy trình**: module thứ nhất **sinh ra bảng công việc** từ một đoạn mô tả hoặc một tệp báo cáo; module thứ hai **gợi ý ai nên làm** từng công việc đó, dựa trên lịch sử làm việc của nhóm.

Module thứ nhất có dùng mô hình ngôn ngữ lớn nhưng **ràng buộc nó bằng cấu trúc dữ liệu**. Module thứ hai **không dùng mô hình ngôn ngữ**, toàn bộ thuật toán tự cài đặt để giải thích được từng con số.

## 1. Module sinh bảng công việc từ mô tả

**Làm gì.** Người dùng dán một đoạn mô tả công việc bằng tiếng Việt, hoặc tải lên tệp `.docx` / `.pdf` (biên bản họp, kế hoạch). Hệ thống sinh ra một bảng hoàn chỉnh: các danh sách, các thẻ công việc, nhãn phân loại, checklist và ngày bắt đầu — hạn chót cho từng thẻ. Người dùng xem trước, sửa lại, rồi mới bấm tạo.

**Vấn đề khó nằm ở đâu.** Ba thứ: mô hình ngôn ngữ hay **bịa ra công việc không có trong văn bản**; hay **tính sai ngày tháng**; và hay **trả về JSON hỏng** khiến tính năng chết hẳn. Ngoài ra văn bản người dùng dán vào có thể chứa câu lệnh nhằm chiếm quyền điều khiển mô hình.

**Giải quyết kiểu gì.**

1. **Bắt mô hình khai số dòng nguồn cho mỗi thẻ.** Thẻ nào không trỏ về được một dòng có thật trong văn bản thì bị loại và được đếm lại. Nhờ vậy "ảo giác" từ một lời phàn nàn chung chung trở thành **một con số đo được**.
2. **Ràng buộc bằng cấu trúc, không bằng lời dặn.** Lược đồ dữ liệu gửi cho mô hình **không có ô nào** để điền ngày tuyệt đối, mã định danh hay mã màu — chỉ có tiêu đề, số dòng nguồn, số ngày lệch và mã màu dạng số. Dù văn bản đầu vào có chiếm được quyền điều khiển mô hình thì nó **cũng không có chỗ để gây hại**.
3. **Bộ luật thắng mô hình về ngày tháng.** Ngày do bộ quy tắc tiếng Việt nhận diện, mô hình không được ghi đè. Mỗi thẻ mang theo nguồn gốc của ngày để luôn trả lời được câu hỏi "ngày này từ đâu ra".

4. **Ép định dạng JSON theo lược đồ, hỏng thì hạ ba mức.** Và mọi lỗi — thiếu khoá, hết giờ, mất mạng, JSON hỏng — đều lùi về kết quả của bộ luật. Hệ thống **không bao giờ báo lỗi dịch vụ**, chạy được ngay cả khi chưa có khoá API.
5. **Màn xem trước bắt buộc.** Không bao giờ tự tạo bảng; người dùng tick, bỏ tick, sửa tiêu đề và ngày rồi mới xác nhận.

**Dùng gì.** Google Gemini, gọi qua một lớp trung lập nhà cung cấp nên đổi sang dịch vụ khác chỉ cần đổi ba biến môi trường mà không sửa mã. Bộ quy tắc nhận diện ngày tiếng Việt viết tất định bằng biểu thức chính quy. Thư viện Zod kiểm tra dữ liệu đầu ra của mô hình. Thư viện mammoth đọc `.docx`, pdf-parse đọc `.pdf`. Toàn bộ nhật ký chạy lưu vào **một bảng cơ sở dữ liệu duy nhất**.

**Kết quả đo được** (25 mô tả tiếng Việt tự soạn, mỗi mẫu chạy ba lần):

| Chỉ số | Chỉ dùng bộ luật | Chỉ dùng AI | Kết hợp (sản phẩm) |
|---|---|---|---|
| F1 | 0,95 | 0,97 | 0,99 |
| Ngày ghi rõ tìm đúng | 100% | 88,6% | 100% |
| Số lần trả JSON hỏng | Không áp dụng | 5/75 (6,7%) | 0/75 |
| Thẻ bịa ra, không có trong văn bản | 0% | 0% | 0% |

Ý chính rút ra: **ép cấu trúc thắng ở độ tin cậy chứ không chỉ ở chất lượng** — khi không ép lược đồ, cứ khoảng 15 lần gọi thì có một lần mô hình trả về rác và người dùng nhận một kế hoạch rỗng.

## 2. Module gợi ý phân công công việc

**Làm gì.** Mở một thẻ công việc, hệ thống xếp hạng các thành viên theo độ phù hợp, mỗi người kèm điểm số, ba thành phần tạo nên điểm đó, và **bằng chứng là đúng những thẻ cũ đã dùng để tính**. Ngoài ra có màn chia việc cho cả một danh sách cùng lúc.

**Vấn đề khó nằm ở đâu.** Hệ thống mới nên **chưa có dữ liệu thật**. Không được bắt người dùng tự khai kỹ năng vì như vậy không còn là cá nhân hoá từ dữ liệu. Mọi con số phải **giải thích được** trước hội đồng và trước trưởng nhóm. Và nếu cứ chọn người giỏi nhất thì **mọi việc sẽ dồn vào một người**.

**Giải quyết kiểu gì.**

1. **Học từ lịch sử có sẵn trong hệ thống.** Mỗi công việc được biến thành véc-tơ từ khoá bằng TF-IDF; hồ sơ của mỗi người chính là các thẻ họ đã hoàn thành; chấm điểm bằng cách lấy **5 thẻ cũ giống nhất**, thẻ càng cũ thì càng nhẹ (nửa đời 90 ngày).

2. **Ba thành phần điểm**: kinh nghiệm chủ đề, độ tin cậy (tỉ lệ đúng hạn — nhưng **tính riêng cho đúng loại việc đang xét**, không phải một chỉ số chung), và khả dụng (số việc đang làm chồng lấn thời gian).
3. **Chuẩn hoá ba thành phần trong nhóm ứng viên** trước khi cộng, vì chúng không cùng thang đo — nếu cộng thô thì bộ trọng số đặt ra sẽ mất hết ý nghĩa.
4. **Thiếu dữ liệu thì nói thiếu.** Người chưa có lịch sử được gắn cờ và bỏ thành phần đó ra khỏi phép tính, chứ **không bị chấm 0 điểm**.
5. **Trọng số của mỗi nhóm tự học từ phản hồi.** Khi hệ thống xếp A đứng đầu mà trưởng nhóm chọn B, trọng số được cập nhật theo hướng đó. Chỉ bắt đầu học từ lượt thứ 10, và lưu lịch sử mọi lần đổi để **vẽ được đường hội tụ**.
6. **Chia việc cả danh sách bằng vòng lặp tham lam**: xếp việc gấp hạn trước, giao xong thì cộng ngay vào tải của người vừa nhận, nên người đó tự tụt hạng ở các việc sau. **Cân tải xảy ra mà không cần thêm tham số nào.**

**Dùng gì.** TypeScript thuần ngay trong máy chủ, **không thư viện học máy, không Python**. Lý do: toàn bộ chỉ là số học cơ bản khoảng 250 dòng, chạy dưới một giây; thêm Python là thêm một dịch vụ trong Docker, thêm một chỗ hỏng mà không thêm năng lực; và quan trọng nhất là giữ được khả năng giải thích từng con số. Thuật toán học trọng số là **perceptron xếp hạng theo cặp**, cùng họ với RankNet. Vì chưa có dữ liệu thật nên em viết một **bộ sinh dữ liệu mô phỏng** có kỹ năng ẩn, người mới, người học nghề và nhiễu để đánh giá.

**Kết quả đo được** (20 bộ dữ liệu mô phỏng, khoảng tin cậy bootstrap 95%):

| Cách giao việc | Xác suất hoàn thành đúng hạn | Mức dồn việc vào một người |
|---|---|---|
| Ngẫu nhiên | 0,389 | 23,2% |
| Chia vòng tròn | 0,404 | 19,9% |
| Chọn người rảnh nhất | 0,395 | 25,9% |
| Chỉ xét kinh nghiệm | 0,397 | 51,7% |
| **Đủ ba thành phần (sản phẩm)** | **0,441** | 34,4% |
| Tham chiếu: lựa chọn tối ưu | 0,611 | 29,9% |

Module thắng cả **5 trên 5 phép so sánh được chọn trước khi chạy thí nghiệm**, và đứng đầu ở **13 trên 13 thế giới mô phỏng** khác nhau. Riêng phần chia việc cả danh sách giảm mức dồn việc từ 49,7% xuống 35,3% đồng thời vẫn tăng tỉ lệ đúng hạn.

## 3. Ba ý xuyên suốt cả hai module

1. **Mọi con số phải truy vết được về dữ liệu có thật.** Module một bắt mỗi thẻ trỏ về một dòng có thật; module hai có nút "Vì sao?" mở ra đúng những thẻ cũ đã tạo nên điểm số.

2. **Ràng buộc bằng cấu trúc, không bằng lời dặn.** Đây là cách chống cả ảo giác lẫn tấn công chèn lệnh mà không phụ thuộc vào việc mô hình có "nghe lời" hay không.
3. **Máy đề xuất, người quyết định.** Không màn hình nào tự ghi vào cơ sở dữ liệu; và hệ thống tự thu số liệu từ hành vi thật (tỉ lệ chấp nhận, số lần chỉnh sửa) để đánh giá sau này.

## 4. Điều cần nói thẳng về hạn chế

Cả hai module được đánh giá trên **dữ liệu do chính em tạo ra** — 25 mô tả tự soạn và một bộ sinh mô phỏng. Các số liệu chứng minh thuật toán **hoạt động đúng như thiết kế và tốt hơn các phương án nền trong phạm vi dữ liệu đó**, nhưng chưa chứng minh được hiệu quả với người dùng thật. Em đã cài sẵn cơ chế thu số liệu từ hành vi thật, nên khi có người dùng thật thì chỉ cần chạy truy vấn chứ không phải làm lại thí nghiệm.